AI Lab: Adversarial Graph Search & Pathfinding Visualization

Comparative Implementation of Local, Informed, and Optimal Search on Statically Blocked Networks

Algorithms: Beam Search, Hill Climbing, Branch and Bound with Dynamic Programming | **Environment:** Python 3.x & Interactive Web UI

1. Lab Overview & Objectives

In this lab, you will explore how different artificial intelligence search strategies navigate a graph where edges can be statically obstructed.

The graph implementation provided in the professor's AI training GitHub repository already provides:

- A graph representation.
- Management of graph edges whose status parameter (up or down) is already settled.
- The frontend integration required to visualize the graph and the resulting search path.

Your core task is to implement three distinct search algorithms from scratch, handle graph partitions and dead ends, and return the computed path in a format compatible with the existing frontend integration.

2. Required Algorithms to Implement

Beam Search

- A heuristic-based search that explores a limited number of the most promising nodes at each level.
- The beam width controls how many candidate nodes are retained during the search. You should ensure that the algorithm can handle situations where all available candidates lead to blocked or previously explored portions of the graph.

Hill Climbing Search

- A greedy local search that repeatedly moves toward a neighboring node that appears closer to the goal according to the available heuristic.
- You will observe firsthand how Hill Climbing can:
  - Get stuck in local minima.
  - Reach dead ends.
  - Fail to reach the target even when another valid path exists.

Branch and Bound with Dynamic Programming

- A cost-based search that explores alternative paths while maintaining the best known cost to each node.
- Dynamic programming should be used to avoid unnecessarily recomputing solutions to previously explored states.
- The implementation should be able to determine when the goal is unreachable and terminate cleanly rather than continuing indefinitely.

3. Core AI Challenge: Completeness vs. Local Traps

Because the graph can contain red/blocked edges, some configurations can partition the graph and make the target unreachable.

The implementation must therefore:

- Detect when a graph is unsolvable.
- Avoid infinite loops.
- Avoid crashing when the target cannot be reached.
- Include test cases containing broken edges to verify all of the above behaviors.

The algorithms should be tested not only on successful paths but also on graphs where the target is unreachable because of blocked edges.

4. Required Implementation Contract

All three search algorithms must follow the same return format so that they can integrate seamlessly with the existing frontend.

Every class must implement a to_dict() method that returns a dictionary containing the computed "path". Your code should follow the convention already established by the BFS implementation and other search algorithms in the GitHub repository, particularly for:

- Path construction.
- Tracking the nodes that form the solution path.
- The implementation of the to_dict() function.
- Formatting the result for frontend visualization.

For example:

```
def to_dict(self):
    return {
        "path": self.path
    }
```

5. Testing Requirements

Each algorithm must be tested on multiple graph configurations, including:

- **Test Case 1 — Valid Path:** A graph where the target is reachable and the algorithm successfully returns a path.
- **Test Case 2 — Broken Edges:** A graph containing one or more edges whose status is down. The algorithm must correctly avoid those edges.
- **Test Case 3 — Unsolvable Graph:** A graph where the broken edges completely separate the start node from the target. The algorithm must terminate, return an empty path, not enter an infinite loop, and not raise an unexpected exception.
- **Test Case 4 — Search-Specific Failure:** Where applicable, include a configuration demonstrating the specific limitations of the search strategy, such as:
  - Beam Search discarding a branch that could eventually lead to the goal.
  - Hill Climbing becoming trapped in a local minimum.
  - Branch and Bound exploring alternative paths while maintaining the best known cost.

6. Grading & Deliverables

You must submit:

- A modular Python backend.
- All three search algorithms: Beam Search, Hill Climbing, Branch and Bound with Dynamic Programming.
- Frontend rendering integration.

You may use the frontend visualization provided in the professor's GitHub repository, or implement your own visualization method, provided that the resulting search path is accurately rendered.

The grading evaluates code correctness, handling of unsolvable partitions and broken edges, proper algorithm implementation, dynamic programming usage, and accurate path visualization.

■ **Important Policy on Generative AI & Academic Integrity**

Generative AI Usage Note: You are permitted to use generative AI tools as an aid in writing your code. However, if you choose to do so, you **must explicitly mention it** in your submission. The LaTeX report template will be provided to you. You can load the template into Overleaf (https://www.overleaf.com/) and modify it directly to complete your report.

Regarding the accompanying PDF report: **it must be entirely written by you.** Generative AI can *only* be used for minor grammar or spelling corrections. Your report must explicitly document:
- Which AI tools were used.
- What specific parts of the code the AI helped generate.
- What you learned from the process.
- What you changed or corrected from the AI's output.
- The breakdown of the algorithm logic.
- Your testing process and results.
- Any limitations or mistakes you discovered.

Defense Session: You will be required to defend both your PDF report and your code during an interactive grading session.

In Short: The assignment asks you to build:
Search Algorithm → **Path/Failure Result** → **Frontend Visualization**
using **Beam Search + Hill Climbing + Branch and Bound with Dynamic Programming** while ensuring correct handling of reachable and unreachable targets in a graph with statically blocked edges.